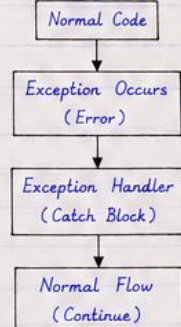


1. Exception Handling (एक्सेप्शन हैंडलिंग) :

(A) Definition of Exception Handling (परिभाषा) :

- Exception Handling एक mechanism है जो Runtime पर होने वाली असामान्य स्थितियों (abnormal conditions) को handle करता है।
- जब program execution के दौरान कोई error आता है जिसे normal code से handle नहीं किया जा सकता, तब एक exception generate होता है।
- Exception Handling program को crash होने से बचाती है और उसे gracefully terminate करने में मदद करती है।
- Java में exception handling का use करके हम error को detect, handle और recover कर सकते हैं।

How Exception Handling Works?



(B) Implementation of Keywords: try, catch, finally

1. try :

- try block में वह code लिखा जाता है जिससे exception आने की संभावना होती है।
- यदि try block में कोई exception occur होता है, तो control सीधे catch block में चला जाता है।

Syntax : try

```
try {  
    // risky code  
}  
catch (ExceptionType e) {  
    // handle exception  
}  
finally {  
    // always executed  
}
```

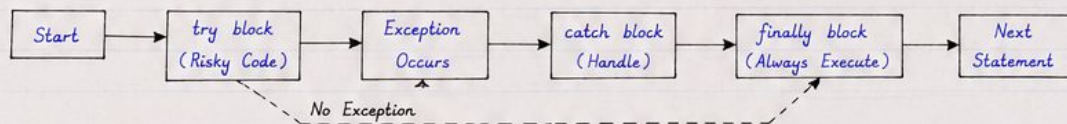
2. catch :

- catch block उस exception को handle करता है जो try block में occur हुआ था।
- एक try block के साथ एक या एक से अधिक catch block हो सकते हैं।

3. finally :

- finally block हमेशा execute होता है, चाहे exception आए या नहीं।
- इसका use resources को release करने (जैसे file close, connection close) के लिए किया जाता है।

Flow of Exception Handling :



Example Program :

```
class DemoException {  
    public static void main(String[] args) {  
        int a = 10, b = 0, result;  
        try {  
            result = a / b; // risky code  
            System.out.println("Result = " + result);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero.");  
        }  
        finally {  
            System.out.println("Finally block executed.");  
        }  
        System.out.println("Program Ended.");  
    }  
}
```

Output :

Cannot divide by zero.
Finally block executed.
Program Ended.

Multiple catch Example :

```
try {  
    int arr[] = new int[5];  
    arr[5] = 10 / 0;  
} catch (ArithmeticException e) {  
    System.out.println("Arithmetic Exception");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Index Out of Bounds");  
} finally {  
    System.out.println("Finally block");  
}
```

Key Points :

- ✓ try block के बिना catch या finally नहीं लिख सकते।
- ✓ finally block हमेशा execute होता है।
- ✓ आप एक try के साथ multiple catch लिख सकते हैं लेकिन catch का order specific होना चाहिए (specific exception पहले, general exception बाद में)।
- ✓ Exception Handling से program की reliability और robustness बढ़ती है।

1. Exception Handling (एक्सेप्शन हैंडलिंग) :

(A) throw & throws :

1. throw :

- throw keyword का use किसी specific exception को explicitly throw करने के लिए किया जाता है।
- यह केवल method के अंदर use होता है।
- throw करने के बाद, control तुरंत nearest catch block में चला जाता है।

Example :

```
void validate(int age) {  
    if (age < 18) {  
        throw new ArithmeticException("Age must be 18 or more");  
    }  
    System.out.println("Age is valid");  
}  
public static void main(String[] args) {  
    validate(15);  
}
```

← यहाँ throw करके exception explicitly भेजा गया है।

2. throws :

- throws keyword का use method declaration में किया जाता है।
- यह declare करता है कि method के अंदर होने वाले exception को caller method को pass (propagate) किया जाएगा।
- throws multiple exceptions declare कर सकता है।

Example :

```
void readFile() throws IOException {  
    // code that may throw IOException  
}  
void display() throws IOException, SQLException {  
    // code  
}
```

- यहाँ method readFile() और display() अपने caller को exception forward कर देंगे।

throw vs throws

	throw	throws
1. Use	Exception को explicitly throw करने के लिए	Method declaration में exception declare करने के लिए
2. Location	Method के अंदर	Method declaration के साथ
3. Purpose	Specific exception को throw करता है	Exception को caller method को pass करता है
4. Control	Control तुरंत catch block में चला जाता है	Control caller method को transfer हो जाता है
5. Example	throw new IOException("Error occurred");	void read() throws IOException

(B) Built-in Exceptions in Java :

- Java में exceptions को दो categories में divide किया गया है :

1. Checked Exceptions (Compile time check)
2. Unchecked Exceptions (Runtime exceptions)

- कुछ commonly used built-in exceptions नीचे दिए गए हैं :

Category	Exception Class	Description	Example
Checked Exceptions	IOException	Input/Output operations के दौरान error होने पर।	File not found, network error
	SQLException	Database access या SQL operations में error होने पर।	Invalid query, connection failure
	ClassNotFoundException	जब required class load नहीं हो पाती।	Class.forName("MyClass")
	FileNotFoundException	जब file specified location पर नहीं मिलती।	new FileInputStream("file.txt")
	InterruptedException	जब thread को wait या sleep से interrupt किया जाता है।	Thread.sleep(1000)
Unchecked Exceptions	ArithmeticException	Arithmetic operation में error (जैसे divide by zero)।	int x = 10/0;
	NullPointerException	जब किसी null object को use किया जाता है।	obj.length();
	ArrayIndexOutOfBoundsException	जब array index उसकी range से बाहर हो।	arr[10] जब size 5 हो
	NumberFormatException	जब String को number में convert नहीं किया जा सकता।	Integer.parseInt("abc");
	ClassCastException	जब incompatible types के बीच cast किया जाता है।	String s = (String) obj;
	IllegalArgumentException	जब method को illegal या invalid argument दिया जाता है।	new Thread(null);
	IllegalStateException	जब method call object के current state के लिए गलत है।	Scanner.nextLine(); (बिना hasNextLine())

Key Points :

- ✓ throw से हम manually exception create करके throw करते हैं।
- ✓ throws से हम method के साथ exception declare करते हैं।
- ✓ Checked exceptions को handle करना जरूरी है (try-catch या throws के द्वारा)।
- ✓ Unchecked exceptions (Runtime) program को crash कर सकती हैं अगर handle न की जाएं।

1. Exception Handling (एक्सेप्शन हैंडलिंग) :

(A) Creating Own Exception Sub Classes (अपनी Exception Sub Classes बनाना) :

- कभी-कभी built-in exceptions हमारे requirement के अनुसार sufficient नहीं होते।
- ऐसी स्थिति में हम अपनी custom exception classes create कर सकते हैं।
- इसके लिए हम Exception class को extend करके अपनी sub class बनाते हैं।

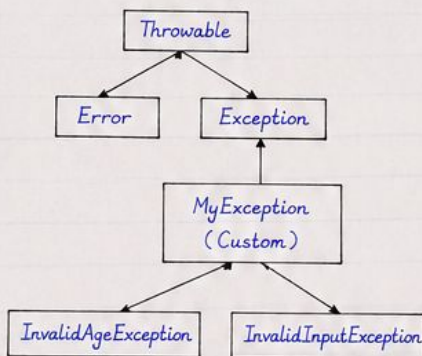
Syntax :

```
class MyException extends Exception {  
    MyException(String message) {  
        super(message);  
    }  
}
```

Example :

```
class InvalidAgeException extends Exception {  
    InvalidAgeException(String msg) {  
        super(msg);  
    }  
}  
  
class Test {  
    static void checkAge(int age) throws InvalidAgeException {  
        if (age < 18) {  
            throw new InvalidAgeException("Age must be 18 or more.");  
        }  
        else {  
            System.out.println("Age is valid.");  
        }  
    }  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
        } catch (InvalidAgeException e) {  
            System.out.println("Exception: " + e.getMessage());  
        }  
    }  
}
```

Custom Exception Hierarchy :



(B) Importance of Exception Handling in Practical Implementation of Live Projects :

1. **Program Reliability :** Exception handling se program crash होने से बचता है और normal flow maintain रहता है।
2. **User Friendly Application :** Proper error messages देकर user को सही information दी जा सकती है।
3. **Data Integrity :** गलत data या invalid operations से data corrupt होने से बचता है।
4. **Resource Management :** Files, Database connections, Network resources को properly close कराया जा सकता है जिसे memory leak या resource leak नहीं होता।
5. **Debugging आसान :** Specific exceptions पकड़कर problem की root cause identify करना आसान हो जाता है।
6. **Code Maintainability :** Structured exception handling से code clean, readable और maintain करने योग्य बनता है।
7. **Security :** Sensitive operations (जैसे file access, DB access) में exceptions handle करके unauthorized access या गलत input से system को सुरक्षित रखा जा सकता है।
8. **Scalability :** बड़े applications (Live Projects) में unexpected conditions को gracefully handle करके application stable और scalable बनाई जा सकती है।

Practical Implementation in Live Projects (उदाहरण) :

Scenario	Possible Exception	Why Handling is Important	Handling Strategy
User Login	InvalidInputException	Invalid input से login fail हो सकता है।	Input validate करें, user को message दें।
File Upload	FileNotFoundException, IOException	File ना मिलने या read/write error से application crash हो सकता है।	File existence check करें, try-catch से handle करें, user को notify करें।
Database Access	SQLException	Wrong query या connection failure से system बंद हो सकता है।	Connection check, try-catch, rollback और उचित message दें।
Payment Processing	InsufficientBalanceException	Insufficient balance पर transaction fail हो सकता है।	Custom exception throw करके user को सही reason बताएं।
Network Communication	SocketException	Network issues की वजह से data transfer fail हो सकता है।	Retry mechanism, exception handle करके connection re-establish करें।

Key Points :

- ✓ हम अपनी जरूरत के अनुसार custom exception sub classes बना सकते हैं।
- ✓ Exception handling live projects में stability, reliability और security के लिए अत्यंत आवश्यक है।
- ✓ Try-catch, finally और custom exceptions का सही उपयोग professional applications की quality बढ़ाता है।